RUNI – Algorithms – Homework 4

Due date: Sunday 07/12/2025

NOTE: the general homework guidelines published in homework 1 are valid for all homework assignments in this course.

Problem 1 (20 pts):

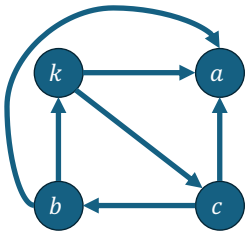
Let $G = (V, E)$ be a directed graph, $w: E \to \mathbb{R}$ a weight function on the edges such that no negative cycle exists in G. Assume that $d(u, v)$ is known for all $u, v \in V$, and provided in a matrix form so that $d(u, v)$ can be accessed in $O(1)$ by reading $d[u][v]$. Let $e = (a, b)$ with given weight $w(a, b)$ be an edge that does **not** exist in E, and let $G' = (V, E \cup e)$.

a. Describe an algorithm that computes the shortest path distances $d(u, v)$ in graph G' for all $u, v \in V$ in $O(|V|^2)$ time.

b. Prove the correctness of your algorithm.

Problem 2 (20 pts):

Recall that a **tournament graph** is a simple directed graph where the underlying undirected graph is a clique. Let $G = (V, E)$ be a tournament graph. A vertex $k \in V$ is called a ***king*** if for every other vertex $v \in V$ there is a directed path of length at most 2 from k to v. In the diagram there is an example tournament graph over 4 vertices, with 3 kings (k, b, c).



a. Claim True/False and shortly Justify: The DFS run starting at a king vertex will create a single tree.

b. Claim True/False and shortly Justify: Any BFS run of a tournament graph G that has a king vertex will yield a forest with a single tree.

c. Provide a simple algorithm that given a vertex $u \in V$ of G runs in time $O(m)$, where $m = |E|$, and correctly decides if u is a king of G or not. shortly justify correctness and analyze runtime.

It has been shown that every tournament graph is guaranteed to have at least one king vertex.

d. Provide a simple algorithm that runs in time $O(n^3)$ and finds a king in the graph G.

Let $x \in V$ be some vertex. We say that x *beat* another vertex $y \in V$ if the edge (x, y) exists in the tournament graph's edges E. Let $L_x \subseteq V$ be the set of vertices that x beats, and $W_x \subseteq V$ be the set of vertices that beat x. Note that the out-degree of x is exactly $|L_x|$.

e. Complete the proof of the following claim:

if x has the maximum out-degree of all vertices, then x is a king.

All vertices in L_x are distance 1 from x by definition. We show that all vertices in W_x are distance 2 from x.

Let vertex $u \in W_x$. Assume towards contradiction that there does not exist any $w \in L_x$ with $(w, u) \in E$.

f. Provide an algorithm that runs in time $O(n + m)$ and finds a king in the graph G. State the correctness claim for this algorithm (no need to prove) and shortly analyze its runtime.

Problem 3 (20 pts):

Let $G = (V, E)$ be a **directed acyclic graph** (DAG). Let $s \in V$ and let $w: E \rightarrow \mathbb{R}^+$ be a non-negative weight function. Describe an efficient algorithm to find every for vertex $v \in V$, a path from s with **maximum** weight.

Justify the correctness of your algorithm and analyze its runtime.

Problem 4 (20 pts):

Let $G = (V, E)$ be a directed graph with real weights w: $\boldsymbol{V} \cup \boldsymbol{E} \rightarrow \mathbb{R}$ on both its **edges and its vertices**. The weight of a path $P = s, v_1, \ldots, v_n, t$ is the sum of the weights of its edges **and the weights of its vertices**. Note that the weights of s and t are also included.

Given $G = (V, E), w$ and $s \in V$, suggest an algorithm that computes the shortest path from s to any other vertex $v \in V$. Justify the correctness of your algorithm and analyze its runtime.

The following problems assume knowledge of the Floyd-Warshall algorithm to be taught in week 6.

Problem 5 (20 pts):
Run the Floyd-Warshall algorithm on the graph given below by δ_0. Describe (fill in) δ_i for each step.

δ_0	1	2	3	4	5
1	0	5	1	∞	∞
2	∞	0	∞	-2	∞
3	2	∞	0	∞	∞
4	8	∞	4	0	∞
5	∞	∞	4	2	0

δ_1	1	2	3	4	5
1					
2					
3					
4					
5					

δ_2	1	2	3	4	5
1					
2					
3					
4					
5					

δ_3	1	2	3	4	5
1					
2					
3					
4					
5					

δ_4	1	2	3	4	5
1					
2					
3					
4					
5					

δ_5	1	2	3	4	5
1					
2					
3					
4					
5					